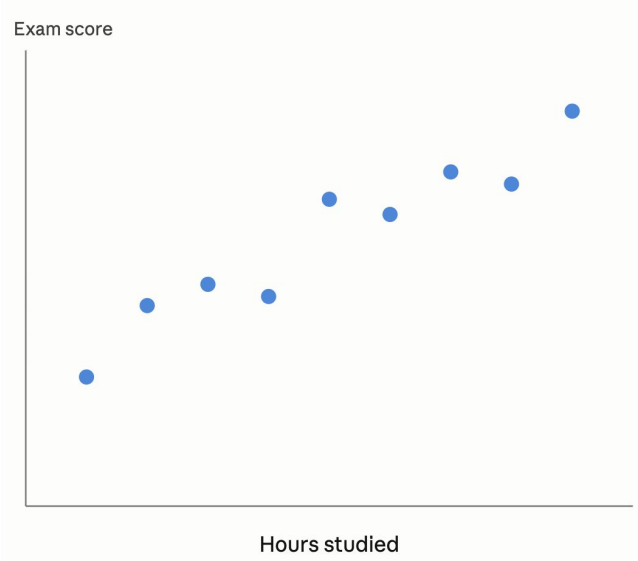


Session 1 - Models and Overfitting

Classical Machine Learning Workshop
by Jonah Sandow

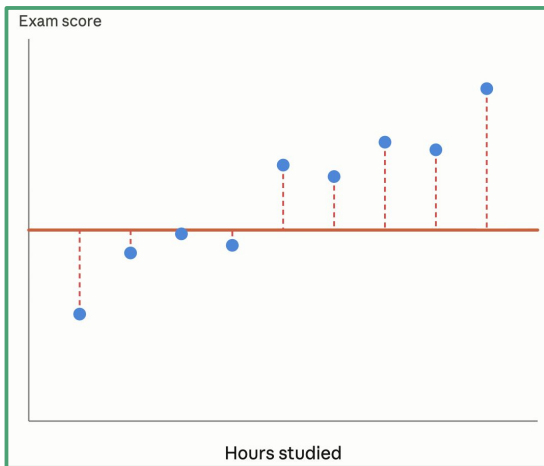
What is a model?

- A linear model...
 - Turns an input into a prediction: Hours studied $\rightarrow$ [model] $\rightarrow$ predicted exam score
 - Learns the rule from examples (the data). We don't write the rules
 - Picks the line that misses the real data the least (minimizes error)

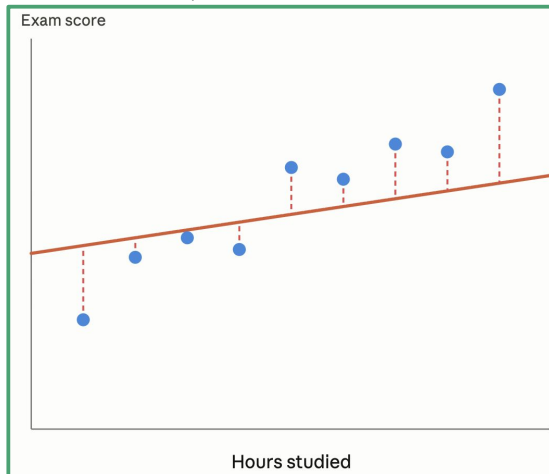


How a model learns (it shrink the mistakes)

1.



2.

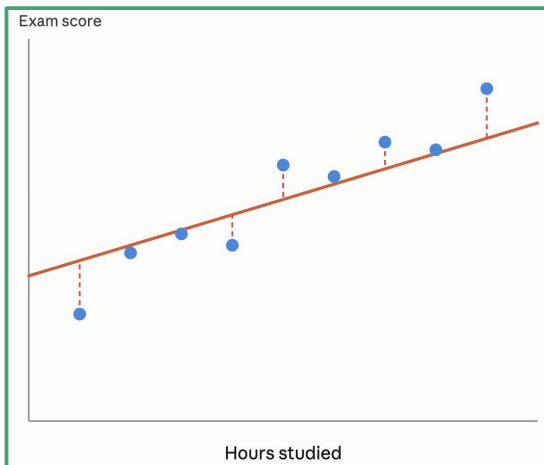


- The data
- The model
- - - The error

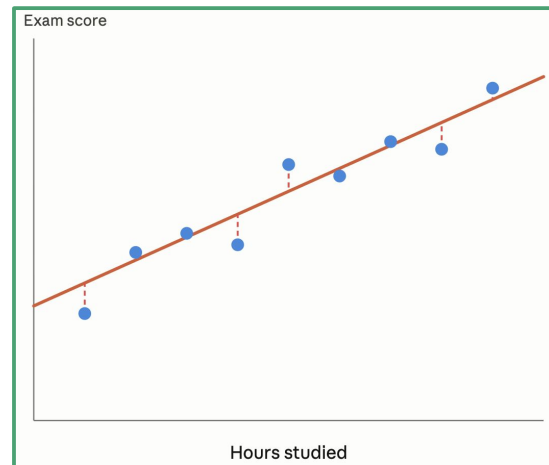
Each round:

Predict -> measure
every miss -> nudge
the line -> repeat

3.



4.



The model learns its:
Weights/Coefficients/ β

$$z = \beta_0 + \beta_1 \cdot x$$

Comprehension Check

1. Does the line need more nudging?
Or is it finished? How can you tell?



2. **Put these steps in order:**

adjust the line, measure the miss, make a prediction, repeat

3. **What actually changes?:** When the model nudges the line, what number inside the model is being updated?"

Comprehension Check

Takeaways:

- Learning = guess -> measure the miss -> adjust -> repeat.
- A straight line can't touch every dot, and that's okay
- When we adjust the line, we are simply adjusting the coefficients (also called the weights or β) of the line - slope and y-intercept

Loss Function

- A loss function is **one number that scores how wrong the model is right now** (lower is better). It's the "total mistake" from the last slide that the model is trying to shrink.
- It's built from the misses: take each prediction's gap from the real value and combine them into a single score.
- Mean Squared Error (MSE) (common loss function for linear regression): square each miss (predicted answer - real answer), then average.
 - Squaring keeps all misses positive (so equally bad misses in opposite directions don't cancel each other out)
 - Squaring also punishes large misses harder
- Training loop: Predict -> compute the loss -> adjust the model to lower the loss -> repeat until the loss can't be lowered anymore

Takeaway: Training = making the loss as small as possible.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \tilde{y}_i)^2$$

Logistic regression: from a line to a probability

- Linear Regression: predict a number (exam score)
- Logistic Regression: predict a category (pass or fail)
 - the same line $\rightarrow$ [squash into 0–1] $\rightarrow$ category
- Perfect for yes/no questions like pass/fail or healthy/sick
- The output is a *probability*; a *threshold* turns it into a decision

Sigmoid

- A straight line shoots below 0 and above 1 (impossible probabilities)
- But a probability has to stay between 0 (definitely no) and 1 (definitely yes)
- So we squash the line into an S-shaped curve (the Sigmoid function)
- Now every output is given to us as a 0-100% chance.

Multiple Features:

- $z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \dots + \beta_n x_n$
- for 20 features, you add 20 terms, the sigmoid stays exactly the same.

Step 1 — the linear score

$$z = \beta_0 + \beta_1 \cdot x$$

Step 2 — the sigmoid

$$P = \text{sigmoid}(z)$$

$$= 1 / (1 + e^{(-z)})$$

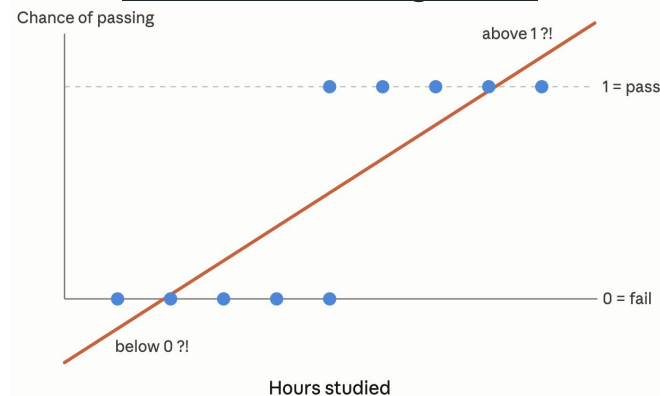
Symbol meanings

z — raw score (any real number)
 β_0 — baseline (score when $x = 0$)
 β_1 — weight (how much x shifts z)
 x — the input (biomarker level)

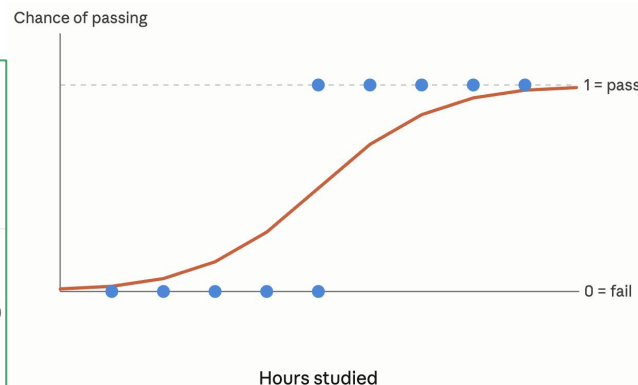
New in step 2

P — probability of disease (0 to 1)
 z — the score from step 1 (same z)
 e — Euler's number ≈ 2.718
 sigmoid squashes any z into 0-1

The broken straight line

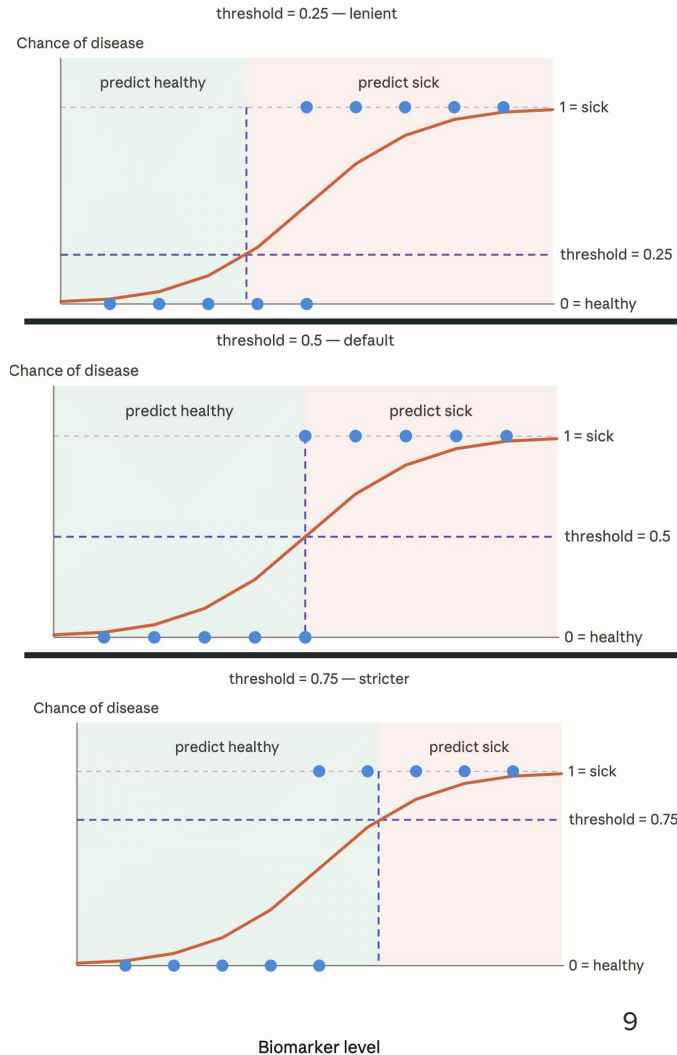


The same data now with an S-curve that stays in bounds



Threshold

- Let's say our model reads some biomarker and outputs sick or healthy
- It now asks: given this patient's biomarker level, what's the probability they have the disease?
- Still the S-curve gives a probability, not a yes/no (or pass/fail)
- So we pick a cutoff (or threshold): above it -> sick, below it -> healthy
- Move the bar up or down and the split moves with it
- Too strict a threshold -> real cases get missed; too lenient -> too many false alarms



Comprehension Check - Threshold

Setup: You're a doctor using a logistic regression model to screen patients for a disease. The model gives you a probability for each patient. You still have to decide: what probability is high enough to send someone for follow-up testing?

Questions:

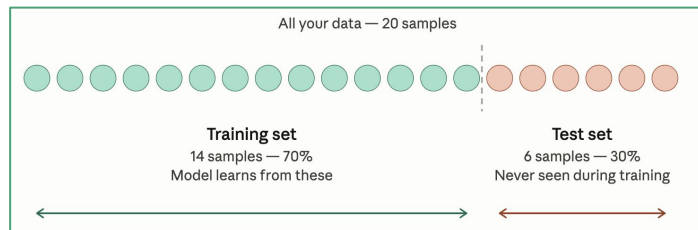
1. Doctor A sets their threshold at 0.4. Doctor B sets theirs at 0.7. Which doctor flags more patients as sick? Which one is more likely to miss a real case?
2. Which doctor would you want if you were the patient? Does your answer change if the disease is easily treatable vs. has no cure?

Takeaway: There's no universally correct threshold. The right threshold depends on which mistake is more costly.

A score on training data is a lie

- The entire goal of ML is to perform well on data the model has never seen (new patients, future samples, tomorrow's experiment)
- A score on training data tells us nothing, the model could simply be memorizing the answers
- So the only honest measure of "will this work in the real world?" is performance on held-out data (data the model never touched during training)
- This is why we split the data into a train and test set. Without a held-out set, you have no instrument to measure what actually matters

Goodhart's Law in ML: When a metric becomes a target, it ceases to be a good measure. If you change your model based on the test set score, that score stops being an honest measure of real-world performance.

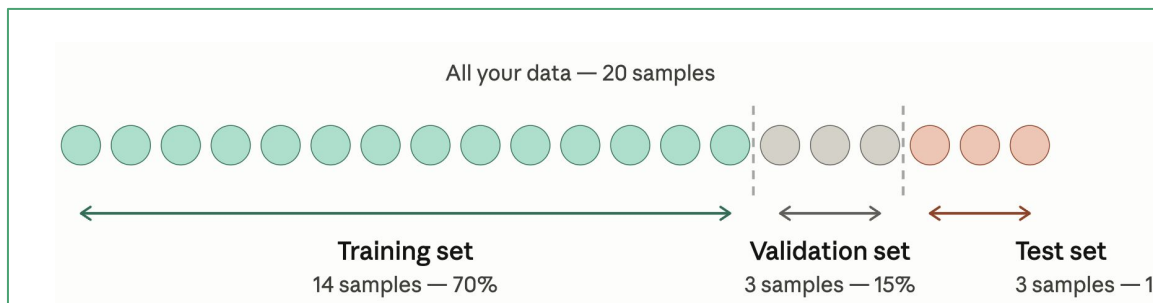


Analogy	Data split	Its job
Homework	Train	model learns its coefficients
Practice exam	Validation	tune the knobs (<i>more soon</i>)
Final exam	Test	graded once — the honest score

Takeaway: Training accuracy flatters. Held-out accuracy is the only score that counts.

Train/Validation/Test Split

- **Training set:** The model learns from this. It's the textbook and the practice problems
- **Validation set:** you use this to make decisions: which model complexity? Which threshold? which settings? Think of it as a practice exam you can retake
- **Test set:** the final exam. Graded once, at the very end, never peeked at before that moment
- Why we need both validation and test sets:
 - Let's say we want to test different model complexities, classification thresholds, or regularization strengths, We you need a third set for this (the validation set)
 - Each data point lands in **exactly one bucket** (no overlap)
 - Typical rough split: **70% train / 15% validation / 15% test** (the exact numbers matter less than the principle)



Comprehension Check - Train/Validate/Test Split

Setup: You have already trained a logistic regression model to predict whether a patient will respond to a new drug. The model outputs probabilities. You want to compare classification thresholds of 0.3, 0.5, and 0.7 and choose the threshold that gives the best performance.

1. **Which set do you use to try the three thresholds and pick the winner?**
 - A) Training set
 - B) Validation set
 - C) Test set
2. **Why can't you use the test set for this?** If you tuned your model using the test set, what goes wrong with your final accuracy number?

Takeaway:

Validation is for tuning. Test is for grading. Using the test set to make decisions corrupts the only honest score you have.

Overfitting

Definition: Overfitting happens when a model learns the training data too well. It memorizes the random noise instead of understanding the general pattern.

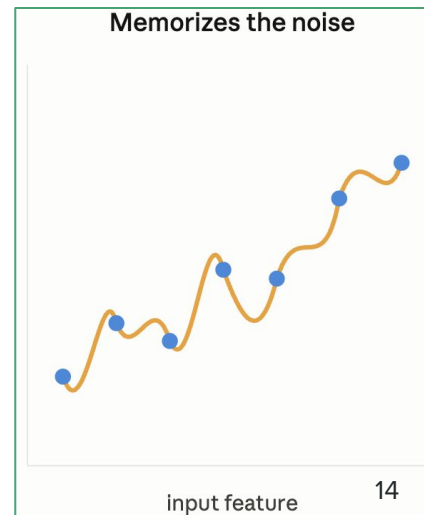
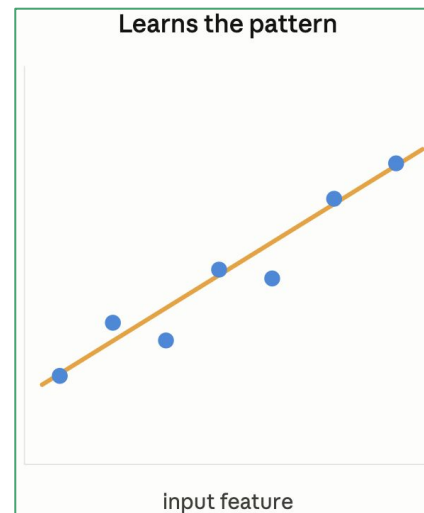
- Imagine fitting a high-degree polynomial (a wildly curvy line) to just a few data points.
- The line perfectly connects every single dot on your graph.

The Warning Signs:

- Training accuracy climbs to near-perfect.
- But there is a gap between training accuracy and validation/test accuracy

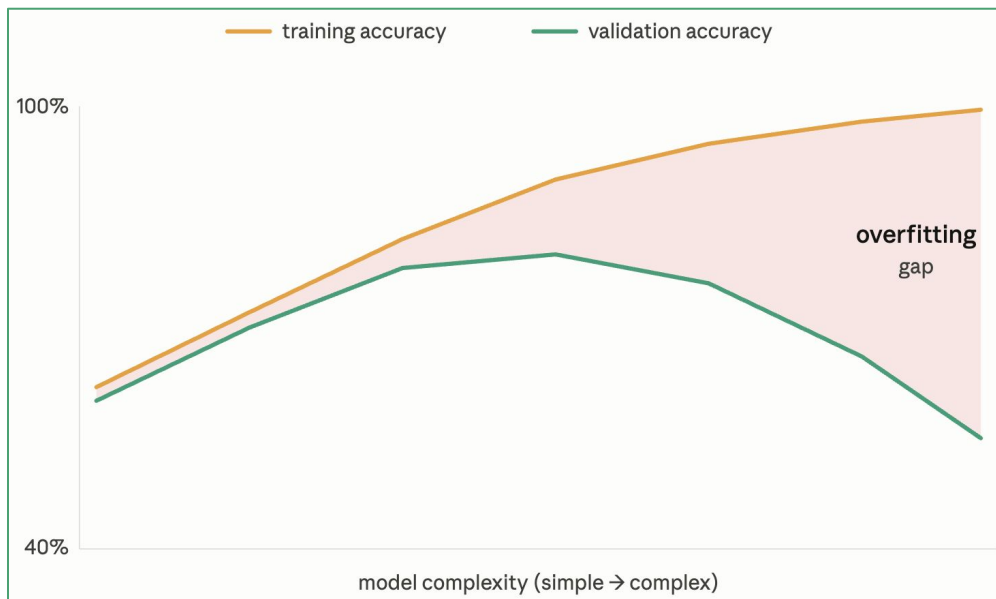
Polynomial Expansion

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \dots + \beta_n x^n$$



Training-Validation Gap

The Takeaway: A massive gap between your training accuracy and validation accuracy is the textbook definition of overfitting. The model looks great in practice, but fails in the real world.



Bias-variance: the two ways to be wrong

The Balancing Act: Every machine learning model tries to find the sweet spot between being too simple and too complex.

High Bias (Underfitting): The model is too simple.

- It completely misses the actual patterns in the data
- example: trying to fit a straight linear regression line to data that is clearly a curve

High Variance (Overfitting): The model is too complex.

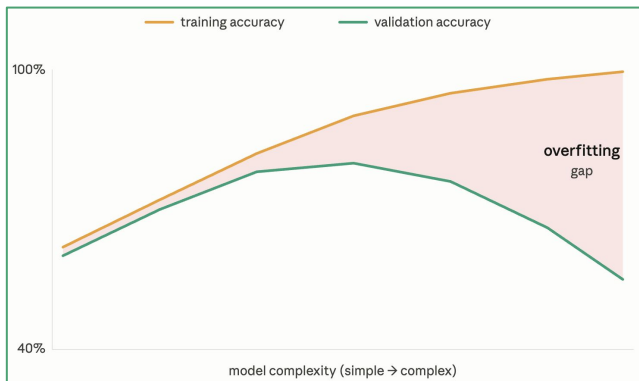
- It chases the noise and fluctuations in the training set, making it highly unstable when given new data.

The Goal: Minimize total error by finding the perfect middle ground where the model is flexible enough to learn, but disciplined enough to generalize.

Comprehension Check - Bias-Variance

For each model below, say whether it's suffering from high bias, high variance, or neither why?

1. A model predicts that every patient in the dataset has a 50% chance of the disease, regardless of their biomarker levels.
2. A model gets 99% accuracy on your training samples but only 58% on new patient data.
3. A model gets 84% on training and 82% on new data.



Takeaway: High bias = too simple, ignores the pattern. High variance = too complex, memorizes the noise. You want the model that generalizes, not the one that just looks good on paper.

left side = high bias zone, right side = high variance zone

The Solution: Regularization

The Definition: A technique used to prevent overfitting by keeping the model's coefficients (weights) in check.

How It Works: It modifies the loss function by adding a "penalty" for big coefficients. If the model wants to give a feature a massive weight to memorize a data point, it gets penalized.

The Role of Lambda (λ): This is the knob you turn to control the penalty strength (tuned using your validation set).

- **High Lambda:** Heavy penalty, forcing weights closer to zero (keeps the model simple).
- **Zero Lambda:** No penalty, leaving you with a standard, unregularized model prone to overfitting.

$$\text{Total Loss} = \text{Loss function} + \lambda(\text{regularization penalty})$$

Regularization Techniques

Ridge - Squared Penalty (aka L2):

- **The Penalty:** Penalizes the *squared* values of the coefficients.
- **The Behavior:** It shrinks all coefficients toward zero, but it rarely makes them exactly zero.
- **Best Use Case:** When you have many features, and you think most of them contribute at least a little bit to the prediction.
- **Total loss** = loss function + $\lambda \sum (\beta_j)^2$

Lasso - Absolute Value Penalty (aka L1):

- **The Penalty:** Penalizes the *absolute* values of the coefficients.
- **The Behavior:** It drives the weights of useless features to **exactly zero**.
- **Best Use Case:** It acts as automatic **feature selection**. Perfect when you have a ton of features but suspect only a few actually matter.
- **Total loss** = loss function + $\lambda \sum |\beta_j|$

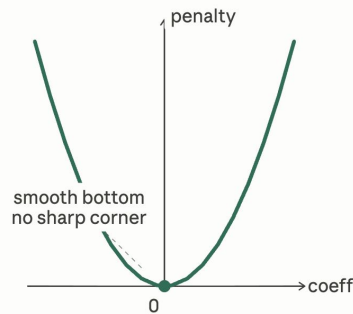
Elastic Net (The Hybrid):

- Combines both L1 and L2 penalties to get the best of both worlds
- **Total loss** = loss function + $\lambda_1 \sum |\beta_j| + \lambda_2 \sum (\beta_j)^2$

In scikit-learn: $C = 1 / \lambda$

Ridge (L2)

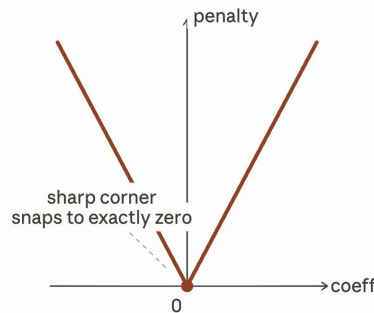
Penalty = coefficient²



Shrinks all coefficients — none reach zero

Lasso (L1)

Penalty = |coefficient|



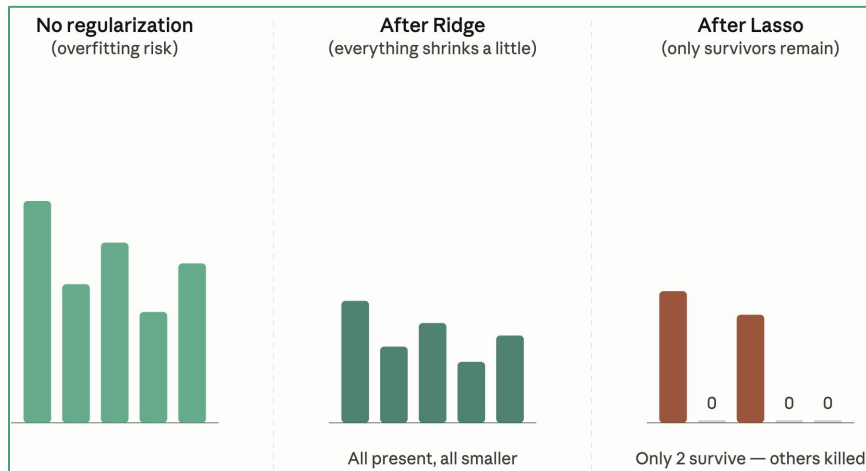
Drives useless coefficients to exactly zero

Comprehension Check - Regularization

You're building a model to predict disease risk from a panel of 500 gene expression measurements. You suspect that only about 10–20 of those genes actually matter.

1. **Which regularization method should you use — Ridge or Lasso? Why?**
2. **You increase lambda (λ) very high.** What happens to the model's coefficients? What happens to training accuracy? What happens to validation accuracy?
3. **Your colleague sets lambda to zero.** What regularization are they applying? What problem might come back?

Takeaway: Ridge shrinks everything a little. Lasso zeros out the irrelevant features entirely (it does automatic feature selection). Use Lasso when you think only a few features really matter.



Thank You!

Any Questions?

Special thanks to Dr. Hau Xu, Marianna Lamanna, and Dr. Joseph Loscalzo for project guidance.

Review

- **A model learns its own rules from data by shrinking its mistakes:** we give it examples and a way to measure how wrong it is, not the answer.
- **The goal is doing well on data it has never seen, not memorizing the data it trained on:** which is why we keep a separate test set and never grade on the training set.
- **More complexity is not always better:** a model that's too complex starts memorizing random noise and falls apart on new data. That's overfitting.
- **The gap between training and validation performance is how you see overfitting:** looks perfect on training, much worse on new data.
- **There are two ways to be wrong: too simple (misses the pattern) or too complex (chases the noise):** the job is to find the middle.
- **Regularization helps fix overfitting by penalizing complexity:** it makes the model pay a price for large weights in the loss function, so it stays simple enough to generalize.

Terminology

- **Model** — a rule that turns an input into a prediction, learned from examples rather than written by hand.
- **Linear model** — the simplest kind of model: it fits a straight line to the data.
- **Coefficients (weights, β)** — the adjustable numbers that define the line; training means finding their best values.
- **Error** — how far a single prediction misses the true value.
- **Loss function** — one number that scores how wrong the model is overall; lower is better.
- **Training** — the repeating loop of measuring the loss and nudging the model to make that number smaller.
- **Logistic regression** — a model that predicts a category (like sick vs. healthy) instead of a number.
- **Sigmoid** — the S-shaped curve that squashes any score into a sensible 0–100% probability.
- **Threshold** — the cutoff that turns a probability into a yes/no decision; move it and the split moves with it.
- **Train set** — the data the model learns from (its homework).
- **Validation set** — held-out data used to tune your choices, like a practice exam.
- **Test set** — held-out data graded exactly once, the only honest measure of real-world performance.
- **Generalization** — doing well on new, unseen data, which is the whole point of a model.

Terminology Continued

- **Overfitting** — when a model memorizes the training data's noise instead of the real pattern, so it looks great in training but fails on new data.
- **Bias (underfitting)** — when a model is too simple and misses the real pattern.
- **Variance (overfitting)** — when a model is too complex and chases random noise.
- **Bias–variance tradeoff** — finding the sweet spot between too simple and too complex.
- **Regularization** — adding a penalty to the loss for large weights, so the model can't memorize.
- **Lambda (λ)** — the knob that controls how strong that penalty is, tuned on the validation set.
- **Ridge** — regularization that shrinks all weights toward zero but rarely to exactly zero.
- **Lasso** — regularization that pushes useless features' weights all the way to zero, doing automatic feature selection.
- **Elastic Net** — a blend of Ridge and Lasso that gets the benefits of both.

Index

1. What is a model: pages 2-6
2. Logistic Regression: pages 7-10
3. Train/Validate/test split: pages 11-13
4. Overfitting: pages 14-15
5. Bias-variance: page 16-17
6. Regularization: pages 18-20
7. Review: page 21
8. Terminology: pages 22-23